

Python と Haskell は競合しない

－ 実験・可視化・統計処理と、型・モデル・意味構造の役割分担 －

Hiroki Funashima

Kindai University Technical College

Sunday, April 26, 2026

1 この文書の狙い

本ノートの目的は、Python と Haskell を単純な優劣関係ではなく、異なる役割を持つ道具として整理することである。

Python は、現代の研究・教育・実務において非常に強力な言語である。特に、データの前処理、可視化、統計処理、機械学習の実験、ファイル操作、外部ライブラリとの連携に強い。研究の現場で「とりあえずデータを読み、加工し、図を出し、実験する」には、Python はきわめて有効である。

一方で、Haskell は別の種類の強みを持つ。Haskell は、型、純粋関数、代数的データ型、パターンマッチ、型クラス、Maybe、Either、IO などを通して、概念の構造や計算の意味をコード上に明示しやすい。そのため、ゲーム理論、意思決定理論、形式的な状態遷移、パーサ、DSL、型による概念整理といった領域で力を発揮する。

したがって、Python と Haskell は競合するというより、役割が異なる。このノートでは、その違いを次の観点から整理する。

- Python が強い領域
- Haskell が強い領域
- 両者の思想の違い
- 研究ワークフローにおける役割分担
- ゲーム理論・意思決定理論・TDA・ネットワーク解析への接続

本ノートは、Python を否定して Haskell を称揚するためのものではない。むしろ、Python の強さを認めた上で、Haskell が担うべき別の役割を明確化するためのメモである。

2 設計思想の違いから見る Python と Haskell

Python と Haskell の役割分担を考える前に、両者の設計思想の違いを確認しておく必要がある。なぜなら、言語が得意とする作業は、単にライブラリの数や実行速度だけで決まるのではなく、その言語が何を自然な書き方として促すかによって大きく左右されるからである。

Python は、読みやすさ、実用性、柔軟性、既存システムとの接続を重視する言語である。プログラマが現実の問題に素早く取り組み、データを読み、加工し、外部ライブラリを呼び出し、結果を確認することを容易にする。この意味で、Python は現実の作業と接続するための言語である。

一方、Haskell は、純粋関数、静的型付け、型推論、代数的データ型、遅延評価、型クラスなどを通して、計算の構造を明示することを重視する言語である。Haskell では、値の形、失敗可能性、副作用、状態遷移、抽象化の境界が型や関数の形として表れやすい。この意味で、Haskell は概念構造を記述しやすい言語である。

したがって、Python と Haskell の違いは、単なる構文やライブラリの違いではない。両者は、プログラムを何のために書くのかについて異なる重心を持っている。

観点	Python	Haskell
重視するもの	実用性・可読性・柔軟性	純粋性・型・意味構造
自然な使い方	動かしながら試す	定義しながら整理する
強い局面	接続・実験・可視化	構造化・仕様化・概念整理
失敗の扱い	例外・None・慣習	Maybe, Either などの型
副作用	通常の処理に混在しやすい	IO として明示される
抽象化	クラス・関数・duck typing	型・型クラス・代数的データ型

この設計思想の違いを踏まえると、Python がデータ処理、可視化、統計処理、機械学習実験に強く、Haskell がゲーム理論、意思決定理論、状態遷移、パーサ、DSL、型による概念整理に強いという分担は、かなり自然に見えてくる。

3 Python の設計思想：現実の作業に素早く接続する

Python の設計思想を一言で言えば、実用的で読みやすく、現実の作業に素早く接続できることである。Python のコードは、英語に近い読みやすさを持ち、比較的少ない記述で多くの処理を書ける。また、動的型付けにより、探索的なコードを書きやすい。

例えば、CSV ファイルを読み、列を選び、平均を取り、図を描くという作業は、Python では非常に短く書ける。

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("experiment.csv")
summary = df.groupby("method")["score"].mean()
summary.plot(kind="bar")
plt.show()
```

このコードでは、厳密な型定義やデータ構造の設計よりも、現実のデータを素早く読み、変換し、結果を見ることが重視されている。これは Python の強さである。

Python は、現実のデータ、既存ライブラリ、ファイル、Web API、Notebook、可視化ツールと接続するための言語として非常に優れている。研究の初期段階では、何が重要かまだ分からないことが多い。その段階では、まず動かし、見て、試すことが重要になる。Python はそのような探索的作業に適している。

ただし、この柔軟性は、概念の境界を曖昧にしやすいという側面も持つ。例えば、同じ `float` が確率、効用、距離、重み、閾値のいずれを表しているのかは、変数名や文脈に依存しやすい。また、関数が `None` を返す可能性があるのか、例外を投げるのか、入力の中の条件で失敗するのかは、型だけでは分かりにくい。

もちろん、Python でも型ヒント、`dataclass`、`pydantic`、`mypy` や `pyright` などを用いれば、かなり構造的なコードを書くことはできる。したがって、Python が構造化できないというわけではない。ただし、その構造化は言語全体の標準的な実行モデルに強制されるというより、ライブラリやツール、書き手の設計方針によって支えられる部分が多い。

つまり、Python は実験と接続には非常に強いが、概念の境界や意味構造の明示は、書き手がどれだけ意識的に設計するか依存しやすい。

4 Haskell の設計思想：概念構造を型と関数で明示する

Haskell の設計思想を一言で言えば、計算を純粋な関数と型によって構造化することである。Haskell では、値は基本的に不変であり、関数は入力から出力を作る写像として扱われる。また、副作用は IO として型に現れ、失敗可能性は Maybe や Either として表現できる。

例えば、文字列をゲームとして解釈する関数を考える。

```
parseGameText :: String -> Either String Game
```

この型を見れば、この関数は文字列を受け取り、成功すれば Game を返し、失敗すれば String のエラーメッセージを返すことが分かる。

さらに、ファイルからゲームを読み込む関数なら、次のようになる。

```
readGameFile :: FilePath -> IO (Either String Game)
```

ここでは、ファイル入出力という副作用が IO に現れ、その結果としてパースの成功または失敗が Either に現れている。このように、Haskell では、関数が何を行い、どこで失敗し、副作用を持つかが型に表れやすい。

また、Haskell では、同じ実行時表現を持つ値でも、概念が異なれば型として分けることができる。

```
newtype Probability = Probability { unProbability :: Double }
    deriving (Eq, Ord, Show)

newtype Utility = Utility { unUtility :: Double }
    deriving (Eq, Ord, Show)

newtype Distance = Distance { unDistance :: Double }
    deriving (Eq, Ord, Show)
```

このようにすれば、確率、効用、距離はいずれも内部的には Double で表されるとしても、概念としては別の型になる。これは、数理モデルや研究コードでは重要である。なぜなら、研究コードにおけるバグの多くは、単なる構文ミスではなく、概念の取り違いや暗黙の前提の混入から生じるからである。

Haskell は、こうした概念の違いをコード上に明示することを促す。そのため、Haskell は「早く試す」ための言語というより、「何を扱っているのかを明確にする」ための言語として強い。

5 設計思想の違いが役割分担を生む

Python と Haskell の違いは、単なる機能比較ではない。両者の設計思想の違いが、そのまま役割分担の違いにつながっている。

Python は、現実のデータや外部ライブラリに接続し、素早く実験することを容易にする。そのため、データの前処理、可視化、統計処理、機械学習の実験、探索的分析に向いている。

Haskell は、型と関数によって概念構造を明示することを容易にする。そのため、ゲーム理論、意思決定理論、形式的な状態遷移、パーサ、DSL、型による概念整理に向いている。

この違いは、次のように整理できる。

問い	Python が向く場合	Haskell が向く場合
まず何が起きているか見たい	データを読み、図を描く	主目的ではない
既存ライブラリを使いたい	非常に向いている	ライブラリ次第
概念の境界を明示したい	書き方に依存する	型で表しやすい
失敗可能性を設計に含めたい	例外・慣習に依存しやすい	Maybe, Either が使える
副作用を分離したい	設計規律が必要	IO として型に出る
DSL を設計したい	可能	構文と意味を分けやすい
長期的な研究資産にしたい	設計次第	型とモジュールで支えやすい

したがって、Python と Haskell のどちらが優れているかを問うのは、あまり生産的ではない。重要なのは、研究や教育のどの局面で、何を明示したいのかである。

データを見たいなら Python がよい。構造を定義したいなら Haskell がよい。実験を回したいなら Python がよい。概念を育てたいなら Haskell がよい。

このように考えると、Python と Haskell は競合しない。むしろ、研究の異なる段階を支える補完的な道具として位置づけることができる。

6 Python の実務上の強さ

まず確認すべきことは、Python が非常に強い言語だということである。特に研究用途において、Python は次のような領域で圧倒的に便利である。

- データ読み込み
- CSV, JSON, Excel, SQLite などとの連携
- データの前処理
- 統計処理
- グラフ描画
- 機械学習の実験
- 数値計算
- Jupyter Notebook による探索的分析
- Web API との連携
- ファイル操作やバッチ処理

Python の強さは、言語そのものだけでなく、ライブラリ生態系にある。例えば、数値計算、データフレーム処理、可視化、機械学習、ネットワーク解析、TDA など、多くの研究作業が Python の豊富なライブラリの上で成立している。

このため、研究の初期段階では、Python は非常に自然な選択になる。仮説を立て、データを読み、簡単な統計量を見て、図を描き、アルゴリズムを試す。このような探索的な作業では、Python の柔軟性と即応性が大きな利点になる。

例えば、データを読み込んで、列を選び、簡単な集計をする作業は、Python では短く書ける。

```
import pandas as pd

df = pd.read_csv("data.csv")
summary = df.groupby("category")["value"].mean()
print(summary)
```

このような作業は、Haskell でも不可能ではない。しかし、Python の方が自然であり、ライブラリも豊富であり、試行錯誤が速い。

したがって、Python は研究の探索段階に非常に向いている。

7 Python の強さは「探索」と「接続」にある

Python の強みを一言で言えば、探索と接続である。

Python は、さまざまなデータ形式、ライブラリ、外部ツール、実験環境を接続するのが得意である。研究では、理論的にきれいな入力だけが与えられるわけではない。むしろ、現実のデータは不完全で、欠損があり、形式が乱れ、前処理を必要とすることが多い。

Python は、そのような現実のデータを扱うための道具として強い。

作業	Python の強み	理由
前処理	非常に強い	pandas などの豊富な道具がある
可視化	非常に強い	グラフ描画ライブラリが豊富
機械学習実験	非常に強い	主要ライブラリが集中している
ファイル処理	強い	スクリプトとして書きやすい
API 連携	強い	Web・JSON・HTTP との親和性が高い
探索的分析	非常に強い	Jupyter Notebook と相性がよい

Python は、現実の雑多な対象を取り込み、実験可能な形に整え、すぐに結果を確認するための言語である。この点で、Python は研究の外界との接点を担う。

言い換えるなら、Python は「データに触れる手」として優れている。データを読み、削り、整え、描き、試す。このような作業を迅速に進めるには、Python はきわめて強い。

8 構造化の道具としての Haskell

Python がこれほど強いなら、Haskell は何を担うのか。この問いに対する答えは、Haskell は実験の即応性よりも、概念の明示性に強いということである。

Haskell は、次のような対象を扱うときに力を発揮する。

- 数理モデルの構造
- 状態と遷移
- 入力形式と意味構造
- 成功と失敗の区別
- 副作用の有無
- 概念間の境界
- DSL の構文と意味
- アルゴリズムの意味的分解

Haskell では、関数の型に多くの情報が現れる。

例えば、

```
parseGameText :: String -> Either String Game
```

という型を見れば、この関数は文字列を受け取り、成功すれば `Game` を返し、失敗すれば `String` のエラーメッセージを返すことが分かる。

また、

```
readGameFile :: FilePath -> IO (Either String Game)
```

という型を見れば、この関数はファイル入出力を行い、その結果としてパース成功または失敗を返すことが分かる。

Python でも同じことは書ける。しかし、Haskell ではそれが型に現れる。この違いは大きい。

Haskell は、プログラムの挙動だけでなく、プログラムが扱う概念構造を明示しやすい言語である。

9 Python は実験の言語、Haskell は構造の言語

大まかに言えば、Python は実験の言語であり、Haskell は構造の言語である。

もちろん、これは単純化である。Python でも構造的なコードは書けるし、Haskell でも実験的なコードは書ける。しかし、言語が自然に促す方向性には違いがある。

観点	Python	Haskell
得意な局面	探索・実験・可視化	構造化・型による整理・意味の明示
主な関心	データをどう処理するか	概念をどう表現するか
失敗の扱い	例外・None・慣習	Maybe, Either などの型
副作用	通常の処理に混在しやすい	IO として明示される
抽象化	クラス・関数・duck typing	型・型クラス・代数的データ型
実験速度	高い	用途による
意味構造の明示	書き方次第	言語機能として強い

Python は、動かしながら理解するのに向いている。Haskell は、定義しながら理解するのに向いている。

Python は、「まずデータを触ってみる」ことを容易にする。Haskell は、「そもそも何を対象としているのか」を明示することを促す。

この違いは、研究において非常に重要である。研究には探索も必要だが、同時に概念の整理も必要だからである。

10 データ前処理・可視化・統計処理では Python が強い

データ前処理や可視化では、Python が非常に強い。これは Haskell が劣っているというより、Python の生態系がその用途に非常に適しているということである。

例えば、CSV を読み、欠損値を処理し、列を変換し、グラフを描くという作業を考える。Python では、これを短いスクリプトや Notebook で進められる。

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("experiment.csv")
df = df.dropna()
df["score"] = df["score"].astype(float)

df.groupby("method")["score"].mean().plot(kind="bar")
plt.show()
```

このような作業では、型による厳密な概念整理よりも、実際のデータを素早く扱うことが重要になる。列名を確認し、欠損値を眺め、外れ値を見つけ、図を出す。Python はこのような探索に向いている。

特に Jupyter Notebook は、研究の初期段階で強力である。コード、出力、グラフ、メモを一つの場所に置けるため、試行錯誤の履歴を残しやすい。

したがって、実験データを扱う局面では、Python を使うのが自然である。

11 機械学習の実験では Python が強い

機械学習の実験でも Python は非常に強い。主要なライブラリやチュートリアル、サンプルコード、周辺ツールが Python を中心に整備されているからである。

例えば、データセットを読み込み、モデルを訓練し、評価指標を出す作業は Python で行うのが自然である。

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.ensemble import RandomForestClassifier

X_train, X_test, y_train, y_test = train_test_split(X, y)

model = RandomForestClassifier()
model.fit(X_train, y_train)

pred = model.predict(X_test)
print(accuracy_score(y_test, pred))
```

このような実験では、ライブラリの充実度、チュートリアルの多さ、周辺ツールとの連携が重要である。その点で Python は非常に強い。

Haskell でも機械学習的な処理を行うことは可能である。しかし、研究や教育の現実的なワークフローでは、Python を使う方が速い場合が多い。

したがって、機械学習の実験そのものを Haskell へ無理に移す必要はない。むしろ、Python で実験し、Haskell ではその背後にあるモデルや状態遷移、仕様、構造を整理する、という分担が自然である。

12 ゲーム理論では Haskell が何を与えるか

ゲーム理論では、Haskell の強みがかなりはっきり現れる。なぜなら、ゲーム理論は概念の構造が非常に重要な分野だからである。

例えば、標準形ゲームでは、少なくとも次の概念が現れる。

- プレイヤー
- 戦略
- 戦略集合
- 利得
- 利得行列
- best response
- Nash equilibrium
- mixed strategy
- support

Python では、これらを辞書、リスト、NumPy 配列、dataclass などで表現できる。

```
payoffs = {
    ("U", "L"): (3, 2),
    ("U", "R"): (1, 1),
    ("D", "L"): (0, 0),
    ("D", "R"): (2, 3),
}
```

これは便利である。しかし、行プレイヤーの戦略と列プレイヤーの戦略の区別、利得の意味、ゲーム全体の整合性は、かなり慣習に依存する。

Haskell では、これらを型として分けやすい。

```
newtype RowStrategy = RowStrategy { unRowStrategy :: String }
    deriving (Eq, Ord, Show)

newtype ColStrategy = ColStrategy { unColStrategy :: String }
    deriving (Eq, Ord, Show)

data Payoff = Payoff
    { payoffP1 :: Double
    , payoffP2 :: Double
    }
    deriving (Eq, Show)

data Cell = Cell
    { cellRowStrategy :: RowStrategy
    , cellColStrategy :: ColStrategy
    , cellPayoff      :: Payoff
    }
    deriving (Eq, Show)
```

```
deriving (Eq, Show)
```

ここでは、`RowStrategy` と `ColStrategy` はどちらも中身は文字列だが、型としては別物である。したがって、行戦略と列戦略を取り違えることを型レベルで防ぎやすくなる。

このように、Haskell はゲーム理論の概念境界をコード上に明示するのに向いている。

13 意思決定理論における Haskell の役割

意思決定理論でも、Haskell は有効である。意思決定理論では、状態、行為、結果、効用、信念、決定規則などの概念を明確に区別する必要がある。

例えば、不確実性下の意思決定では、次のような構造が現れる。

- 状態集合
- 行為集合
- 結果集合
- 効用関数
- 信念または確率分布
- 期待効用
- 決定規則

Python では、これらを柔軟に表現できる。しかし、その柔軟性ゆえに、状態と行為、結果と効用、確率と重みが混ざりやすい。

Haskell では、次のように概念を型として分けることができる。

```
newtype State = State { unState :: String }
    deriving (Eq, Ord, Show)

newtype Act = Act { unAct :: String }
    deriving (Eq, Ord, Show)

newtype Outcome = Outcome { unOutcome :: String }
    deriving (Eq, Ord, Show)

newtype Utility = Utility { unUtility :: Double }
    deriving (Eq, Ord, Show)

newtype Probability = Probability { unProbability :: Double }
    deriving (Eq, Ord, Show)
```

さらに、期待効用を計算する関数の型を明示できる。

```
expectedUtility
:: [(State, Probability)]
-> (Act -> State -> Outcome)
-> (Outcome -> Utility)
-> Act
-> Utility
```

この型を見るだけで、この関数が

- 状態と確率の分布
- 行為と状態から結果を返す関数

- 結果から効用を返す関数
- 評価対象の行為

を受け取り、効用を返すことが分かる。

このように、Haskell の型は、意思決定理論の構造をそのままコード上に写すための枠組みになる。

14 形式的な状態遷移では Haskell が強い

状態遷移を扱う問題でも Haskell は強い。例えば、アルゴリズム、ゲーム木、オートマトン、探索問題、シミュレーションなどでは、状態と遷移を明示することが重要になる。

Python でも状態遷移は書ける。

```
def step(state, action):  
    # update state  
    return new_state
```

しかし、状態の種類、可能な行為、遷移の失敗、終了条件などは、しばしば慣習やドキュメントに依存する。

Haskell では、状態と行為を型で分けることができる。

```
data State  
    = Initial  
    | Running Int  
    | Finished  
    deriving (Eq, Show)  
  
data Action  
    = Start  
    | Advance  
    | Stop  
    deriving (Eq, Show)  
  
step :: State -> Action -> Either String State  
step Initial Start      = Right (Running 0)  
step Initial _          = Left "invalid action from Initial"  
step (Running n) Advance = Right (Running (n + 1))  
step (Running _) Stop    = Right Finished  
step Finished _         = Left "already finished"
```

この例では、状態の形、行為の種類、失敗可能性が型とパターンマッチに現れている。

状態遷移を形式的に扱う場合、Haskell は「何が可能で、何が不可能か」を明示しやすい。これはアルゴリズムの検証、シミュレーション、ゲーム木の解析などに有効である。

15 パーサと DSL では Haskell が強い

Haskell はパーサや DSL と相性がよい。その理由は、入力の構文と意味構造を型として対応させやすいからである。

例えば、ゲーム理論の簡単な入力形式を考える。テキストファイルには、戦略名と利得行列が書かれているとする。

Python では、文字列処理や正規表現で柔軟に処理できる。

```
def parse_game(text):
    lines = text.splitlines()
    # parse rows and payoffs
    return game
```

これは速く書ける。しかし、入力形式が複雑になるにつれて、どこで何を読んでいるのか、失敗時に何を返すのか、読み取った構造が正しいのかが曖昧になりやすい。

Haskell では、パース結果を型として明示できる。

```
parseGameText :: String -> Either String Game
```

また、DSL の抽象構文木を代数的データ型として定義できる。

```
data Expr
  = Var String
  | Number Double
  | Add Expr Expr
  | Mul Expr Expr
  deriving (Eq, Show)
```

このようにすれば、入力文字列を単なる文字列としてではなく、意味を持った構文木として扱える。

DSL では、構文と意味の対応が重要である。Haskell はこの対応を型と関数で明示しやすい。そのため、パーサ、DSL、形式言語、設定ファイル仕様、ゲーム記述言語などに向いている。

16 型による概念整理

Haskell の重要な強みは、型によって概念を整理できることである。

Python では、異なる意味を持つ値が同じ型で表されることが多い。例えば、次の値はすべて float でありうる。

- 確率
- 効用
- 利得
- 距離
- 閾値
- 時間
- 重み

Python では、これらを区別するには、変数名やクラス設計やドキュメントに頼ることが多い。

Haskell では、軽量に区別できる。

```
newtype Probability = Probability { unProbability :: Double }
    deriving (Eq, Ord, Show)

newtype Utility = Utility { unUtility :: Double }
    deriving (Eq, Ord, Show)

newtype Distance = Distance { unDistance :: Double }
    deriving (Eq, Ord, Show)

newtype Threshold = Threshold { unThreshold :: Double }
    deriving (Eq, Ord, Show)
```

このように書くことで、同じ Double で表される値であっても、概念として区別できる。

これは単なる型安全性の問題ではない。研究においては、概念の混同を防ぐことが重要である。Haskell の型は、概念の境界をコード上に刻むための道具になる。

17 TDA・ネットワーク解析における分担

TDA やネットワーク解析では、Python と Haskell の役割分担が特に興味深い。

Python は、データ読み込み、ネットワーク構築、可視化、数値実験に強い。例えば、グラフデータを読み込み、中心性を計算し、可視化する作業は Python が得意である。

```
import networkx as nx

G = nx.read_edgelist("network.edgelist")
centrality = nx.betweenness_centrality(G)
```

また、TDA のライブラリを用いて、点群データから persistent homology を計算し、barcode や persistence diagram を描くことも Python の得意領域である。

一方、Haskell は、次のような構造化に向いている。

- filtration の定義
- simplex, complex, boundary operator の型定義
- グラフから複体への変換規則
- パラメータ付き解析手順の記述
- 解析パイプラインの意味的分解
- ネットワーク上の状態遷移モデル

例えば、単体複体の構造を型で表すことができる。

```
newtype Vertex = Vertex { unVertex :: Int }
    deriving (Eq, Ord, Show)

data Simplex = Simplex
    { simplexVertices :: [Vertex]
    }
    deriving (Eq, Ord, Show)

data FiltrationValue = FiltrationValue Double
    deriving (Eq, Ord, Show)

data FilteredSimplex = FilteredSimplex
    { filteredSimplex :: Simplex
    , filtrationValue :: FiltrationValue
    }
    deriving (Eq, Show)
```

Python は計算と可視化の実務に強い。Haskell は、その背後にある対象の構造や変換規則を明示するのに強い。

したがって、TDA やネットワーク解析では、Python と Haskell を次のように分担できる。

作業	Python	Haskell
データ読み込み	強い	可能
前処理	強い	可能
ネットワーク解析	強い	構造記述に向く
可視化	非常に強い	外部連携が必要
TDA の数値計算	ライブラリ次第で強い	実装可能だが重い
filtration の意味づけ	書き方次第	型で明示しやすい
解析パイプラインの仕様化	可能	向いている

18 研究ワークフローとしての二層構造

Python と Haskell の役割分担を、研究ワークフローとして考えると、二層構造が見えてくる。

層	主な言語	役割
経験的・実験的層	Python	データ処理、可視化、実験、統計処理
意味的・構造的層	Haskell	型、モデル、DSL、状態遷移、仕様化

Python は、観測データや実験結果に近い層を担う。Haskell は、理論モデルや意味構造に近い層を担う。

これは、次のように図式化できる。

```
Real-world data
  |
  v
Python: preprocessing, statistics, visualization, experiments
  |
  v
Research questions and observed patterns
  |
  v
Haskell: typed models, formal structures, DSLs, semantic decomposition
  |
  v
Refined concepts and algorithms
  |
  v
Python: empirical validation and visualization
```

このように、Python と Haskell は一方向の関係ではない。Python で観察された現象を Haskell で構造化し、Haskell で整理されたモデルを Python で再び実験・可視化する。この往復が重要である。

19 Python だけで十分な場合

もちろん、すべての研究コードに Haskell が必要なわけではない。Python だけで十分な場合も多い。例えば、次のような場合は Python だけでよい。

- 一回限りのデータ可視化
- 単純な統計処理
- 既存ライブラリの利用が中心の機械学習実験
- 小規模な前処理スクリプト
- 概念構造よりも結果の確認が重要な作業

このような場面で、無理に Haskell を使う必要はない。むしろ、Haskell を導入することで作業が重くなる場合もある。

道具の選択では、「何を明示したいのか」が重要である。データの傾向を見たいだけなら Python がよい。概念間の関係、状態遷移、失敗可能性、DSL の意味を明示したいなら Haskell が候補になる。

Haskell を使うべきかどうかは、単に「Haskell が好きかどうか」ではなく、その研究コードにおいて型や構造の明示が価値を持つかどうかで判断すべきである。

20 Haskell を使う価値が出る場合

Haskell を使う価値が出るのは、次のような場合である。

- 同じ概念を何度も使う。
- 入力形式が複雑である。
- 失敗条件を明示したい。
- 状態遷移を正確に管理したい。
- 理論モデルをコードとして保存したい。
- 複数のアルゴリズムを比較したい。
- 研究コードを長期的に育てたい。
- 教育用コードと研究用コードを接続したい。

例えば、ゲーム理論のコードを毎年授業で使い、さらに論文や研究メモにも使うなら、Haskell で型を整理する価値がある。標準形ゲーム、展開形ゲーム、混合戦略、均衡概念、パーサ、表示、解析をモジュールとして分ければ、研究資産として育てやすい。

また、意思決定理論のように、状態・行為・結果・効用・信念・規則といった概念が明確に分かれる領域では、Haskell の型による整理が効く。

Haskell は、短期的な作業の速度では Python に劣る場合がある。しかし、長期的に概念を育てる場合には、Haskell の明示性が強みになる。

21 二つの言語をつなぐ考え方

Python と Haskell を併用する場合、両者を完全に統合しようとする必要はない。むしろ、それぞれが得意な成果物を作ると考える方が自然である。

Python 側の成果物は、例えば次のようなものである。

- 前処理済みデータ
- CSV や JSON
- 図表
- 実験結果
- 統計量
- Notebook

Haskell 側の成果物は、例えば次のようなものである。

- 型定義
- パーサ
- DSL
- 状態遷移モデル
- 解析ロジック
- 形式的仕様に近いコード

両者をつなぐ形式としては、CSV、JSON、YAML、テキスト DSL などが考えられる。

例えば、Python で前処理したデータを JSON として出力し、Haskell 側でその構造を型付きで読み込む。または、Haskell で定義した DSL に基づいてゲームや状態遷移を記述し、その解析結果を CSV として出力して Python で可視化する。

このようにすれば、Python と Haskell は互いの弱点を補う関係になる。

22 教育上の意味

Python と Haskell の役割分担は、教育上も意味がある。

Python は、初学者がプログラミングの有用性を実感しやすい。データを読み、グラフを描き、簡単な分析を行うことで、すぐに結果が見える。そのため、導入教育には非常に向いている。

一方で、Python だけでは、型、意味構造、状態遷移、失敗可能性、副作用の分離といった概念が暗黙のままになりやすい。Haskell は、これらを明示的に学ぶための教材になりうる。

例えば、Python では `None` を返す関数を書いた後で、Haskell の `Maybe` を見ると、「値がないかもしれない」という性質を型で表す意味が理解しやすい。

Python で例外処理を学んだ後で、Haskell の `Either` を見ると、「失敗を値として返す」設計の意味が分かりやすい。

Python でクラスや `dataclass` を学んだ後で、Haskell の `data` や `newtype` を見ると、「データの形」と「概念の境界」を分けて考えやすくなる。

したがって、Python と Haskell は教育上も競合しない。Python は実用と実験の入口であり、Haskell は構造と思考の訓練になる。

23 研究メモとしての位置づけ

このノートで述べている Python と Haskell の分担は、単なる言語比較ではない。研究メモの管理や論文の種の育成とも関係している。

研究の初期段階では、Python 的な作業が多い。まずデータを触り、図を描き、仮説を立てる。この段階では、柔軟性が重要である。

しかし、仮説が育ち、概念が見え、繰り返し使う構造が現れてくると、Haskell 的な整理が効いてくる。対象、状態、操作、結果、失敗、入出力を分けて考える必要が出てくるからである。

つまり、研究の成熟度に応じて、適した言語的態度が変わる。

研究段階	主な問い	向いている道具
探索段階	何が起きているか	Python
整理段階	何を対象としているか	Haskell
実験段階	どの程度成り立つか	Python
形式化段階	どの構造として表せるか	Haskell
発表段階	どう見せるか	Python / L ^A T _E X

このように考えると、Python と Haskell は研究の異なる段階を支える道具である。

24 まとめ

本ノートでは、Python と Haskell を競合する言語としてではなく、異なる役割を持つ道具として整理した。

要点は次の通りである。

1. Python は、データ前処理、可視化、統計処理、機械学習実験、探索的分析に強い。
2. Haskell は、型、代数的データ型、純粋関数、Maybe, Either, IO などを通して、概念構造を明示するのに強い。
3. Python は「動かしながら理解する」ための言語として優れている。
4. Haskell は「定義しながら理解する」ための言語として優れている。
5. ゲーム理論、意思決定理論、状態遷移、パーサ、DSL、型による概念整理では、Haskell の明示性が効く。
6. TDA やネットワーク解析では、Python が計算・可視化を担い、Haskell が構造・意味づけ・仕様化を担うという分担が考えられる。
7. 研究ワークフローでは、Python と Haskell を往復させることで、探索と形式化を接続できる。

結局のところ、Python と Haskell の違いは、単なる構文やライブラリ数の違いではない。それは、プログラムをどのようなものとして捉えるかの違いである。

Python は、現実のデータやライブラリと接続し、素早く実験するための言語である。Haskell は、概念、構造、意味、失敗、副作用を明示し、理論的对象をコードとして整理するための言語である。

したがって、両者は競合しない。むしろ、研究においては補完的である。Python が実験の速度を支え、Haskell が概念の精度を支える。この二つを使い分けることで、実験と理論、探索と形式化、データと意味構造を接続しやすくなる。

付録 A Appendix : Python と Haskell の役割分担表

最後に、Python と Haskell の役割分担を表としてまとめる。

作業・対象	Python	Haskell
データ前処理	非常に強い	可能だが主戦場ではない
可視化	非常に強い	外部連携が必要
統計処理	強い	可能
機械学習実験	非常に強い	目的次第
ネットワーク解析	強い	構造定義に向く
TDA 実験	ライブラリ次第で強い	構造記述に向く
ゲーム理論モデル	柔軟に書ける	型で整理しやすい
意思決定理論	書けるが緩い	概念境界を明示しやすい
状態遷移	書ける	型とパターンマッチが効く
パーサ	書ける	非常に相性がよい
DSL	可能	構文と意味を分けやすい
研究メモの実験	強い	目的次第
研究資産化	書き方次第	型とモジュールで支えやすい

付録 B Appendix：使い分けの短い指針

実際にどちらを使うべきか迷った場合、次のように考えるとよい。

- データをすぐに見たいなら Python。
- 図を描きたいなら Python。
- 機械学習ライブラリを使いたいなら Python。
- 入力形式の意味を厳密に扱いたいなら Haskell。
- 状態と遷移を明示したいなら Haskell。
- 失敗可能性を型で表したいなら Haskell。
- 同じ理論構造を長期的に育てたいなら Haskell。
- 実験と可視化は Python、仕様とモデルは Haskell。

この指針は絶対的なものではない。しかし、研究用コードを書くときに、言語選択を感覚ではなく役割から考えるための目安になる。